

Sistemas Operativos

2º Parcial 2C2025 – TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría – Single Choice

- 1- b) Segmentación pura
- 2- c) Reducir el tiempo de traducción de direcciones
- 3- d) Ninguna de las anteriores
- 4- a) Asignación fija y sustitución global
- 5- d) Todas las anteriores
- 6- b) Accesos directos más eficientes
- 7- c) Su inodo
- 8- c) Su creación implica crear una nueva entrada de directorio

Teoría – Desarrollar

1- El sistema operativo controla los archivos abiertos mediante dos estructuras principales:

Tabla de archivos abiertos por proceso:

- Cada proceso mantiene su propia lista con los archivos que tiene abiertos.
- Registra el **descriptor de archivo**, el **modo de apertura**, la **posición actual de lectura/escritura** y una **referencia** a la información global del archivo.

Tabla global de archivos abiertos:

- Registra todos los archivos abiertos en el sistema.
- Contiene los **permisos de acceso**, el **contador de procesos** que lo usan y el **enlace al archivo físico** (por ejemplo, al *inode*).

Ejemplo:

Si dos procesos abren el mismo archivo, ambos crean una entrada en su tabla local apuntando a la misma entrada en la tabla global.

Cuando uno de ellos lo cierra, se elimina su entrada local y el sistema **decrementa el contador de uso** en la tabla global

Si el contador llega a **cero**, el sistema libera los recursos y se cierra el archivo definitivamente.

2-

$DL = \text{Nro segmento} + \text{Nro página} + \text{offset}$

Las estructuras que intervienen en el proceso de traducción son:

- **Tabla de segmentos:** Contiene, para cada segmento, la **dirección base** de su tabla de páginas y su **límite (tamaño)**. Se utiliza para verificar que la dirección sea válida y para ubicar la tabla de páginas correspondiente.
- **Tabla de páginas:** Traduce el número de página dentro del segmento en el **número de frame** físico donde se encuentra cargada.

Este esquema permite que varios procesos apunten a los mismos frames físicos mediante entradas de tabla que referencien las mismas páginas dentro de un segmento compartido (por ejemplo, código o bibliotecas comunes).

Si dos procesos necesitan **compartir un segmento completo**, basta con que las entradas correspondientes en sus tablas de segmentos apunten a la misma tabla de páginas.

De esta manera, se evita la duplicación de información en memoria y se optimiza el uso de los recursos del sistema.

3- El thrashing es un fenómeno que ocurre cuando el sistema operativo dedica **la mayor parte del tiempo a intercambiar páginas entre memoria y disco** en lugar de ejecutar realmente los procesos. Como resultado, la CPU permanece ociosa y el rendimiento global del sistema cae drásticamente.

La causa de este problema reside en una **cantidad de frames insuficiente para satisfacer la localidad temporal de los procesos**, ya sea porque el grado de multiprogramación es demasiado alto (en un caso de sustitución global) o porque la cantidad de frames asignada a un proceso no es suficiente (sustitución local).

A raíz de esto, **aumenta fuertemente la tasa de page faults, disminuyendo la utilización de CPU**. El sistema entero se vuelve lento o casi inoperable, ya que los procesos en el peor de los casos no pueden continuar con su ejecución y el almacenamiento secundario pasa a ser el cuello de botella.

En un sistema con políticas de asignación dinámica y sustitución global de frames:

- Para detectar podemos adoptar diferentes estrategias:
 - **Analizar la frecuencia de page faults** a nivel sistema y, en base a eso, definir cotas inferiores y superiores para delimitar la frecuencia de los PFs dentro de un nivel "saludable"
 - Hacer uso de la técnica de conjunto de trabajo para aproximar la suma de la localidad temporal de cada uno de los procesos dentro de nuestro grado de multiprogramación y, en base a eso, revisar si la cantidad de frames que tenemos disponible es suficiente.
- Si con alguna de las estrategias mencionadas logramos detectar thrashing, deberíamos **disminuir el grado de multiprogramación** para mitigar el problema.

Práctica

1. 22639 (Escritura) – 27443 (Lectura) – 44241 (Lectura) – 15983(Lectura) – 43010 (Escritura) – 20491 (Lectura)

	P11 E	P13 L	P21 L	P7 L	P21 E	P10 L
Frame 1	P11 11	P11 11	P11 11 <–	P11 01	P11 01	P10 10
Frame 2	<–	P13 10	P13 10	P7 10	P7 10	P7 10 <–
Frame 3		<–	P21 10	P21 00 <–	P21 11 <–	P21 01

TP

Pág	Frame	Bit uso	Bit modificado	Bit presencia
7	2	1	0	1
10	1	1	0	1
11	1	0	1	0
13	2	0	0	0
21	3	0	1	1

2. a) $\text{ptrs} \times \text{bloq} = 1 \text{ KiB} / 8 \text{ B} = 128$

$$2 \text{ GiB} / 1 \text{ KiB} = 2^{31} \text{ B} / 2^{10} \text{ B} = 2^{21} \text{ bloques de datos}$$

$$2^{21} / 2^7 = 2^{14} \text{ bloques de punteros (simples)}$$

Tengo solamente 3 punteros indirectos simples por lo que necesito acceder a los dobles también.

$$2^{14} / 2^7 = 2^7 = 128 \text{ bloques de punteros (dobles)}$$

Tengo solamente 2 punteros indirectos dobles por lo que necesito acceder al triple también que direcciona los 126 bloques que faltan

$$\rightarrow \text{Total de bloques} = 2^{21} \text{ BD} + 2^{14} \text{ BPs} + 2^7 \text{ BPd} + 1 \text{ BPt}$$

$$= 2097152 \text{ BD} + 16384 \text{ BPs} + 128 \text{ BPd} + 1 \text{ BPt} = 2113665 \text{ Bloques}$$

3. a) Antes de la migración, los archivos de 10 KiB pesan 32 KiB cada uno debido a fragmentación interna, ocupando en total 32 MiB, por lo que **no queda espacio libre**.

Luego de la migración, el nuevo tamaño de bloque es de 2 KiB, la fragmentación interna en estos archivos desaparece, quedando **22 MiB de espacio libre**.

$$\text{b) Antes de la migración} \rightarrow \text{Tam FAT} = 2^{12} * 1,5 \text{ bytes} = 48 \text{ Kbytes} = \mathbf{6 \text{ KiB}}$$

$$\text{Luego de la migración} \rightarrow \text{Tam FAT} = 2^{16} * 2 \text{ bytes} = 2^{17} \text{ bytes} = \mathbf{128 \text{ KiB}}$$

4. 4096 B entran justo en base **8192** y **55296**, ambas serían respuestas correctas ya que cumplen con el criterio de Best Fit:

Base	Tamaño
0	8192
34816	8192
12288	14336
8192 o 55296	4096

5. Dado que el archivo ocupa 1 MiB en memoria y la misma se encuentra dividida en frames de 1 KiB, el archivo se distribuye en $2^{10} = 1024$ páginas. Por lo tanto, serán necesarios al menos 1024 accesos para realizar la traducción de direcciones lógicas a físicas (uno por cada página, para obtener el número de frame correspondiente), y otros 1024 accesos adicionales para leer los datos almacenados en dichos frames.

Por otro lado, para persistir un archivo de n bloques en FAT son necesarios:

1 acceso para agregar la entrada de directorio para el archivo nuevo (disco)

- n accesos a la tabla FAT (memoria)
- n accesos a bloques de disco

Para saber cuantos bloques son 1 MiB debemos averiguar el tamaño del bloque:

$$\text{Tam. FAT} = (\text{Cant. de entradas}) \times (\text{Tamaño entrada})$$

$$\text{Tam. FAT} = (\text{Cant. bloques}) \times (\text{Tamaño entrada})$$

$$32 \text{ MiB} = (\text{Cant. de entradas}) \times (32 \text{ bits})$$

$$32 \text{ MiB} = (\text{Cant. de entradas}) \times (4 \text{ bytes})$$

$$2^{25} \text{ bytes} = (\text{Cant. de entradas}) \times 2^2 \text{ bytes}$$

$$\text{Cant. de entradas} = 2^{23}$$

En consecuencia:

$$\text{Tamaño de bloque} = 32 \text{ GiB} / 2^{23} = 2^{35} / 2^{23} = 2^{12} = 4 \text{ KiB}$$

De esta manera el archivo de 1MiB tiene:

$$2^{20} / 2^{12} = 2^8 = 256 \text{ bloques}$$

Haciendo el **recuento final** tenemos:

- Acc. a memoria = Traducciones + lectura frames + accesos tabla FAT
 - $1024 + 1024 + 256 = 2304$
- Acc. a disco = Entrada dir + Escritura de bloques
 - $1 + 256 = 257$